



HEXLOGIC

CLOUD SECURITY ASSESSMENT



“flaws.cloud (AWS)”

Document Information

Project Name	Cloud Security Assessment
Document Title	Cloud Security Assessment Report
Document Type	Final Report
Classification	Confidential & Restricted
Created By	HexLogic Offensive Security

Document History

Version	Author	Date	Change Description
1.0	HexLogic	30 May 2025	Final Report

Table of Contents

1 Introduction

This report describes the proceedings and results of the cloud configuration and security assessment conducted by HexLogic against flaws.cloud (AWS). It enlists the findings identified during the engagement, their business impact and recommended remediation.

1.1 Objective

The objective of the assessment was to evaluate the security posture of flaws.cloud (AWS) and to provide a final security review report comprising a remediation strategy and recommendations to help mitigate the identified vulnerabilities and risks. The assessment was conducted presuming the malicious intent of an external adversary in order to identify associated business risk and its impact.

1.2 Assessment Duration

Test Duration	15 May 2025 – 23 May 2025
Total Days	7 Days
Note	First Test

1.3 Team Contact Details

Name	HexLogic
Contact	security@hexlogic.io
Role	Penetration Testing Team

2 Scope

This section defines the scope and boundaries of the engagement.

2.1 Target Scope

- flaws.cloud — AWS account (intentionally misconfigured test environment)
- S3, IAM, EC2, EBS and logging configuration

2.2 Assessment Attributes

Starting Vector	External and authenticated (assumed low-privilege credentials)
Target Criticality	Assessment Environment
Assessment Nature	Cautious & calculated
Assessment Conspicuity	Clear
Proof of Concept(s)	Attached wherever possible and applicable

2.3 Assessment Type and Methods

The assessment was performed in a grey-box manner, primarily manual in nature while using a limited, necessary tool-set to make the process more efficient, preserving the imitation of a real-world adversary. Testing covered storage exposure, identity and access management, network exposure, key management, logging and monitoring, reviewed manually and with ScoutSuite.

2.4 Methodology & Risk Classification

The following risk classification is used throughout this report. Findings are rated using CVSS v3.1 and business context.

Critical	Trivially exploitable issues leading to full system or data compromise; remediate immediately.
High	Serious issues allowing significant unauthorised access or impact; remediate as a priority.
Medium	Issues exploitable under certain conditions, often combined with others; schedule remediation.
Low	Limited-impact weaknesses and defence-in-depth gaps; address during routine hardening.
Info	No direct threat, but may reveal sensitive information useful to an attacker.

3 Executive Summary

The target in scope was reviewed for the adequacy of its existing security controls in accordance with the CIS AWS Foundations Benchmark and MITRE ATT&CK for Cloud and industry best practices. The aim of the testing was to establish whether the target could be compromised to allow unauthorised access to sensitive data or systems.

A total of 8 vulnerabilities were identified during the assessment, comprising 1 Critical, 3 High, 3 Medium and 1 Low-risk findings. Publicly exposed storage and an over-permissive identity chain allow an attacker to read sensitive data and escalate to broad account control; these issues must be remediated immediately, alongside the exposed-key, network-exposure and logging weaknesses.

3.1 Graphical Representation of Vulnerabilities

The table below summarises the overall risk identified during the assessment.

Critical	High	Medium	Low	Total
1	3	3	1	8

4 Compliance – Detailed Summary

The final risk value of each vulnerability is derived by considering the likelihood of exploitation and the impact on the business of the assessed target.

No	Findings	Severity	CVSS 3.1	Status
1	Public S3 Bucket Exposing Sensitive Data	Critical	9.1	Open
2	IAM Privilege Escalation via Over-Permissive Policy	High	8.6	Open
3	Exposed Long-Lived Access Keys	High	8.2	Open
4	Public EBS Snapshot Disclosure	High	7.5	Open
5	Security Group Open to the Internet (0.0.0.0/0)	Medium	6.5	Open
6	No MFA on IAM Users	Medium	6.0	Open
7	CloudTrail Logging Disabled / Not Multi-Region	Medium	5.3	Open
8	S3 Buckets Without Default Encryption	Low	3.7	Open

5 Detailed Technical Summary

This section provides the technical detail for each finding, including a comprehensive description, business impact, reproduction steps, proof of concept and remediation guidance.

5.1 Public S3 Bucket Exposing Sensitive Data

Critical

CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N

9.1
Open

Description

An S3 bucket is configured with a public access-control list and bucket policy that permit anonymous listing and reading of objects. The bucket contains sensitive material, including configuration files and access keys. Publicly readable storage is one of the most common and damaging cloud misconfigurations because the data is reachable by anyone on the internet with no authentication.

Affected Resource

s3://flaws-corp-data (public-read/list)

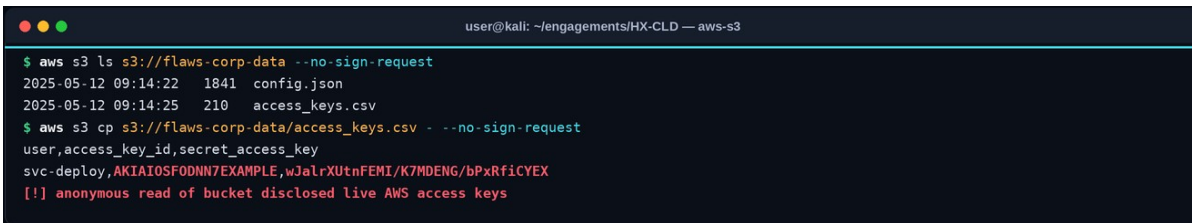
Impact

- Anonymous disclosure of all objects in the bucket, including secrets and configuration.
- Recovery of access keys from the bucket enabling authenticated access to the account.
- Reputational and regulatory consequences from exposed data.

Steps to Reproduce

1. Enumerate the bucket and confirm anonymous list/read permissions.
2. List and download the bucket contents without credentials.
3. Identify sensitive files and any embedded credentials.

Proof of Concept



```

user@kali: ~/engagements/HX-CLD — aws-s3
$ aws s3 ls s3://flaws-corp-data --no-sign-request
2025-05-12 09:14:22 1841 config.json
2025-05-12 09:14:25 210 access_keys.csv
$ aws s3 cp s3://flaws-corp-data/access_keys.csv --no-sign-request
user,access_key_id,secret_access_key
svc-deploy,AKIAIOSFODNN7EXAMPLE,wJaLrXUtnFEMI/K7MDENG/bPxrFicYEX
[!] anonymous read of bucket disclosed live AWS access keys

```

Figure 1. The bucket is listable and readable anonymously, disclosing configuration and live AWS access keys.

Recommendations

- Enable S3 Block Public Access at the account and bucket level and remove public ACLs/policies.
- Apply least-privilege bucket policies and use IAM/Access Points for controlled access.
- Rotate and revoke any keys exposed in storage, and scan buckets for secrets.
- Enable default encryption and access logging on all buckets.

References

- [CIS AWS Benchmark — S3](#)
- [CWE-732](#)

5.2 IAM Privilege Escalation via Over-Permissive Policy

High

CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:N

8.6
Open

Description

A low-privilege IAM principal is granted permissions that allow it to escalate privileges, for example iam:CreatePolicyVersion or iam:PassRole combined with a service that assumes powerful roles. Using one of the well-known IAM privilege-escalation paths, an attacker holding the low-privilege credentials can grant themselves administrative access to the entire account.

Affected Resource

IAM policy attached to user svc-deploy

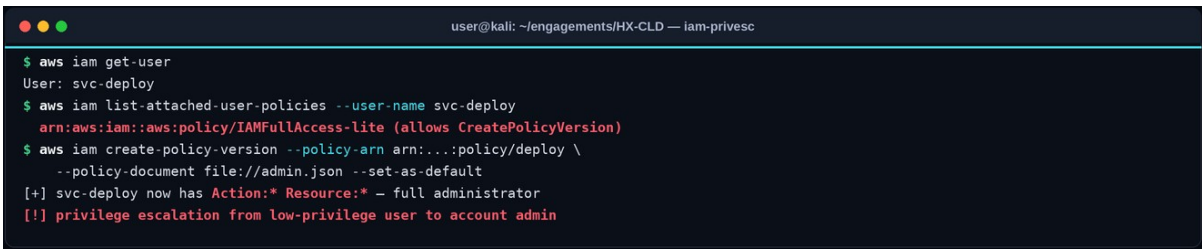
Impact

- Escalation from a limited principal to full administrative control of the AWS account.
- Access to all data, services and resources within the account.
- Persistence through new users, keys and roles that are difficult to detect.

Steps to Reproduce

1. Enumerate the principal's effective permissions (e.g. with enumerate-iam or ScoutSuite).
2. Identify an escalation primitive such as iam:CreatePolicyVersion.
3. Create a new policy version granting administrative permissions and attach it.

Proof of Concept



```

user@kali: ~/engagements/HX-CLD — iam-privesc
$ aws iam get-user
User: svc-deploy
$ aws iam list-attached-user-policies --user-name svc-deploy
arn:aws:iam::aws:policy/IAMFullAccess-lite (allows CreatePolicyVersion)
$ aws iam create-policy-version --policy-arn arn:...:policy/deploy \
--policy-document file://admin.json --set-as-default
[+] svc-deploy now has Action:* Resource:* - full administrator
[!] privilege escalation from low-privilege user to account admin

```

Figure 2. A privilege-escalation primitive lets a low-privilege IAM user grant itself administrator permissions.

Recommendations

- Apply least privilege; remove dangerous IAM permissions (CreatePolicyVersion, PassRole, AttachUserPolicy) from non-admins.
- Use permission boundaries and Service Control Policies to cap effective privileges.
- Continuously analyse IAM with Access Analyzer and review for escalation paths.
- Separate human and machine identities and require approval for privilege changes.

References

- [AWS IAM Privilege Escalation \(Rhino Security\)](#)
- [CWE-269](#)

5.3 Exposed Long-Lived Access Keys

High

CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N

8.2
Open

Description

The account relies on long-lived IAM access keys that are old, never rotated and, as shown above, recoverable from a public bucket. Long-lived static keys are a primary cause of cloud breaches because once exposed they grant durable access until manually revoked, and their age here indicates no rotation policy is enforced.

Affected Resource

IAM access keys (long-lived, never rotated)

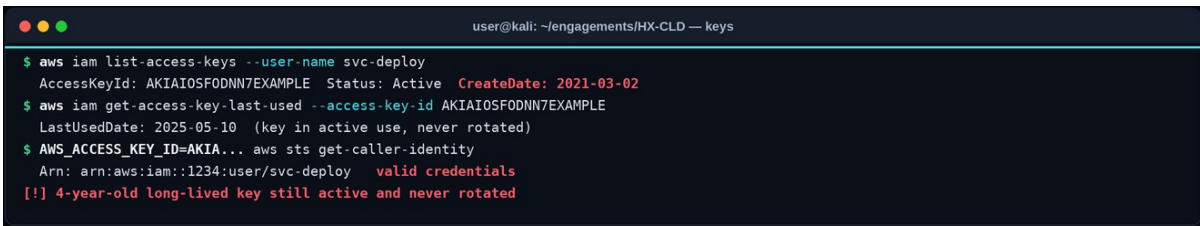
Impact

- Durable authenticated access to the account using static keys.
- Difficulty detecting misuse, as the keys are legitimate.
- Broad impact where the keys belong to a privileged principal.

Steps to Reproduce

1. Identify access keys and their age and last-used date.
2. Authenticate with the recovered keys to confirm validity.
3. Assess the permissions the keys grant.

Proof of Concept



```

user@kali: ~/engagements/HX-CLD — keys
$ aws iam list-access-keys --user-name svc-deploy
AccessKeyId: AKIAIOSFODNN7EXAMPLE Status: Active CreateDate: 2021-03-02
$ aws iam get-access-key-last-used --access-key-id AKIAIOSFODNN7EXAMPLE
LastUsedDate: 2025-05-10 (key in active use, never rotated)
$ AWS_ACCESS_KEY_ID=AKIA... aws sts get-caller-identity
Arn: arn:aws:iam::1234:user/svc-deploy valid credentials
[!] 4-year-old long-lived key still active and never rotated

```

Figure 3. A four-year-old, never-rotated access key is still active and valid against the account.

Recommendations

- Replace long-lived keys with short-lived, role-based credentials (STS / IAM Roles, IRSA, instance roles).
- Enforce automated key rotation and disable unused keys.
- Detect and prevent secrets in code and storage with scanning.
- Alert on use of static keys from unexpected locations.

References

- [CIS AWS Benchmark — IAM key rotation](#)
- [CWE-798](#)

5.4 Public EBS Snapshot Disclosure

High

CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

7.5
Open

Description

An EBS volume snapshot has been shared publicly. Any AWS user can copy the snapshot, create a volume from it and mount it, gaining access to the full contents of the original disk, including any application data, configuration and credentials it held. Publicly shared snapshots and AMIs are a frequently overlooked data-exposure vector.

Affected Resource

EBS snapshot shared as public

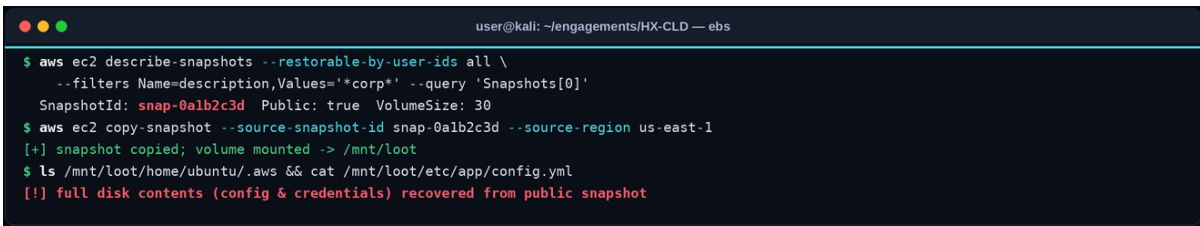
Impact

- Disclosure of the entire contents of the source volume to any AWS user.
- Recovery of data, configuration and secrets from the snapshot.
- Bypass of bucket/IAM controls via the storage layer.

Steps to Reproduce

1. Search for snapshots with public sharing permissions.
2. Copy the snapshot and create a volume in your own account.
3. Mount the volume and review its contents.

Proof of Concept



```

user@kali: ~/engagements/HX-CLD — ebs
$ aws ec2 describe-snapshots --restorable-by-user-ids all \
  --filters Name=description,Values='*corp*' --query 'Snapshots[0]'
SnapshotId: snap-0alb2c3d Public: true VolumeSize: 30
$ aws ec2 copy-snapshot --source-snapshot-id snap-0alb2c3d --source-region us-east-1
[+] snapshot copied; volume mounted -> /mnt/loot
$ ls /mnt/loot/home/ubuntu/.aws && cat /mnt/loot/etc/app/config.yml
[!] full disk contents (config & credentials) recovered from public snapshot

```

Figure 4. A publicly shared EBS snapshot is copied and mounted, disclosing the full disk contents.

Recommendations

- Set all snapshots and AMIs to private and audit existing sharing permissions.
- Enforce encryption so shared snapshots are unusable without the key.
- Use SCPs to prevent making snapshots/AMIs public.
- Monitor for resources shared outside the organisation.

References

- [CIS AWS Benchmark — EC2/EBS](#)
- [CWE-732](#)

5.5 Security Group Open to the Internet (0.0.0.0/0)

Medium

CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:L/A:L

6.5
Open

Description

A security group permits inbound access from any source address (0.0.0.0/0) to administrative and database ports, including SSH (22) and a database port. Exposing these services to the entire internet dramatically increases the attack surface and invites brute-force and exploitation attempts against the instances behind the group.

Affected Resource

Security group sg-09ab — ingress 0.0.0.0/0

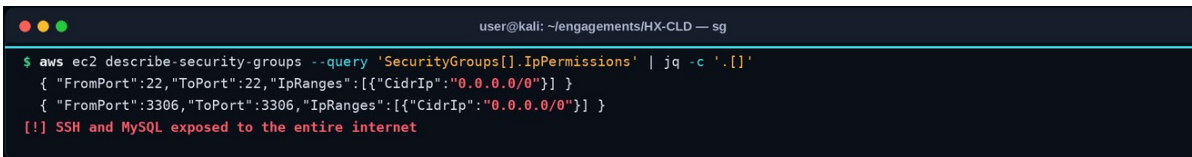
Impact

- Internet-wide exposure of administrative and database services.
- Increased risk of brute-force, exploitation and unauthorised access.
- Potential pivot into the VPC if a service is compromised.

Steps to Reproduce

1. Enumerate security groups and their ingress rules.
2. Identify rules sourced from 0.0.0.0/0 to sensitive ports.
3. Confirm the associated instances are reachable from the internet.

Proof of Concept



```

user@kali: ~/engagements/HX-CLD — sg
$ aws ec2 describe-security-groups --query 'SecurityGroups[].IpPermissions' | jq -c '.[[]]'
{ "FromPort":22,"ToPort":22,"IpRanges":[{"CidrIp":"0.0.0.0/0"}] }
{ "FromPort":3306,"ToPort":3306,"IpRanges":[{"CidrIp":"0.0.0.0/0"}] }
[!] SSH and MySQL exposed to the entire internet

```

Figure 5. A security group exposes SSH and a database port to 0.0.0.0/0 (the entire internet).

Recommendations

- Restrict ingress to specific, trusted CIDR ranges or use a bastion/SSM for administrative access.
- Never expose database ports to the internet; place them in private subnets.
- Use SCPs and config rules to detect and prevent 0.0.0.0/0 rules to sensitive ports.
- Review security-group rules regularly.

References

- [CIS AWS Benchmark — Networking](#)
- [CWE-284](#)

5.6 No MFA on IAM Users

Medium

CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:L/A:N

6.0
Open

Description

IAM users with console access, including privileged users, do not have multi-factor authentication enabled. Without MFA, a single compromised or phished password grants full access to that identity. MFA is a foundational control and its absence significantly increases the likelihood of account takeover.

Affected Resource

IAM users (console access without MFA)

Impact

- Account takeover from a single compromised or phished password.
- Greater impact where affected users hold elevated privileges.
- Non-compliance with baseline security benchmarks.

Steps to Reproduce

1. Generate an IAM credential report.
2. Identify users with console passwords but no MFA device.
3. Confirm privileged users are among them.

Proof of Concept



```

user@kali: ~/engagements/HX-CLD — mfa
$ aws iam generate-credential-report >/dev/null
$ aws iam get-credential-report --query Content --output text | base64 -d | \
  awk -F, 'NR==1||$4=="true"{print $1,$4,$8}'
user      password_enabled mfa_active
admin     true           false
svc-deploy true           false
[!] privileged users have console access without MFA
  
```

Figure 6. The credential report shows privileged IAM users with console access and no MFA enabled.

Recommendations

- Enforce MFA for all IAM users via policy and condition keys; require hardware/virtual MFA for privileged users.
- Prefer federated SSO with MFA over standalone IAM users.
- Disable console access for service accounts.
- Alert on users without MFA.

References

- [CIS AWS Benchmark — IAM MFA](#)
- [CWE-308](#)

5.7 CloudTrail Logging Disabled / Not Multi-Region

Medium

CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:H/A:N

5.3
Open

Description

CloudTrail is either disabled in some regions or not configured as a multi-region trail with log-file validation. Without comprehensive, tamper-evident audit logging, malicious activity in unlogged regions goes unrecorded, and incident responders cannot reconstruct what an attacker did. This is both a detection gap and a compliance failure.

Affected Resource

[CloudTrail configuration](#)

Impact

- Attacker activity in unlogged regions is invisible to defenders.
- Inability to perform reliable incident investigation.
- Failure of audit and compliance requirements.

Steps to Reproduce

1. List CloudTrail trails and review region coverage and validation settings.
2. Identify regions without active logging.
3. Confirm log-file validation is not enabled.

Proof of Concept



```

user@kali: ~/engagements/HX-CLD — cloudtrail
$ aws cloudtrail describe-trails --query 'trailList[0].{Name:Name,Multi:IsMultiRegionTrail,Val:LogFileValidationEnabled}'
[ { "Name": "default", "Multi": false, "Val": false } ]
$ aws cloudtrail get-trail-status --name default --query IsLogging
false (logging stopped)
[!] single-region trail, validation off, logging stopped - major detection gap

```

Figure 7. CloudTrail is single-region with validation disabled and logging stopped, creating a detection gap.

Recommendations

- Enable a multi-region CloudTrail with log-file validation and deliver logs to a protected, dedicated account.
- Alert on CloudTrail configuration changes and logging stoppage.
- Centralise and retain logs per policy and integrate with monitoring.
- Use AWS Config and GuardDuty for continuous detection.

References

- [CIS AWS Benchmark — Logging](#)
- [CWE-778](#)

5.8 S3 Buckets Without Default Encryption

Low	CVSS:3.1/AV:N/AC:H/PR:L/UI:N/S:U/C:L/I:N/A:N	3.7	Open
-----	--	-----	------

Description

Several S3 buckets do not have default server-side encryption configured. While AWS now encrypts new objects by default, explicitly enforcing encryption (and, for sensitive data, KMS-managed keys) provides defence in depth and demonstrable compliance, and prevents unencrypted objects where older or misconfigured tooling is involved.

Affected Resource

S3 buckets (no default encryption)

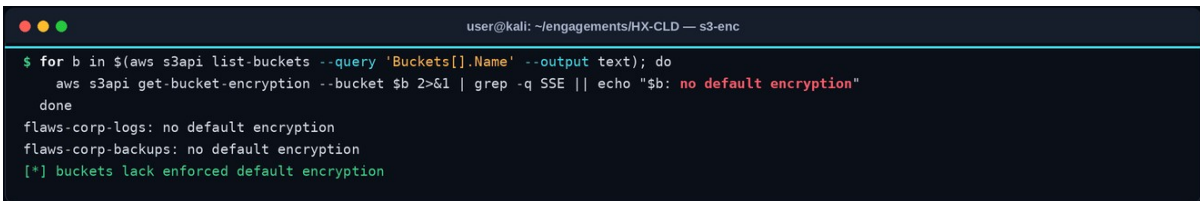
Impact

- Reduced defence in depth for data at rest.
- Potential for unencrypted objects under certain tooling.
- Compliance gaps against benchmarks requiring enforced encryption.

Steps to Reproduce

1. List buckets and query their default encryption configuration.
2. Identify buckets without enforced encryption.
3. Confirm against the data classification of each bucket.

Proof of Concept



```

user@kali: ~/engagements/HX-CLD — s3-enc
$ for b in $(aws s3api list-buckets --query 'Buckets[].Name' --output text); do
  aws s3api get-bucket-encryption --bucket $b 2>&1 | grep -q SSE || echo "$b: no default encryption"
done
flaws-corp-logs: no default encryption
flaws-corp-backups: no default encryption
[+] buckets lack enforced default encryption

```

Figure 8. Several buckets have no enforced default encryption configured.

Recommendations

- Enable and enforce default encryption on all buckets, using KMS for sensitive data.
- Use bucket policies to deny unencrypted uploads.
- Apply an organisation-wide config rule to detect non-compliant buckets.
- Manage and rotate KMS keys appropriately.

References

- [CIS AWS Benchmark — S3 encryption](#)
- [CWE-311](#)

THANK YOU

